Lab Sheet 5 - Solution

SQL Joins

Objective: To practice INNER, OUTER (LEFT, RIGHT, FULL), CROSS, NATURAL and SELF joins, and joins combined with aggregation, in MySQL.

Note: Every query was executed and the output shown is the real result, on a MySQL-compatible server (MariaDB 10.11). The sample data is built so the joins show something interesting: two students have no department (Sneha, Anjali), the Commerce department has no students, the Machine Learning course has no students, and one employee has no manager.

Setup (run first)

Create the database

SQL

```
DROP DATABASE IF EXISTS CollegeDB;
CREATE DATABASE CollegeDB;
```

Output

```
Query OK, 4 rows affected

Query OK, 1 row affected
```

Create tables and load sample data

SQL

```
CREATE TABLE Department (
    Dept_ID   INT PRIMARY KEY,
    Dept_Name VARCHAR(50),
    Location  VARCHAR(50)
);
CREATE TABLE Course (
    Course_ID   INT PRIMARY KEY,
    Course_Name VARCHAR(50),
    Duration    VARCHAR(20),
    Fees        DECIMAL(10,2),
    Dept_ID     INT
);
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name       VARCHAR(50),
    Age        INT,
    City       VARCHAR(30),
    Dept_ID    INT,          -- NULL = not assigned to any department
    Course_ID  INT           -- NULL = no course yet
);
CREATE TABLE Employee (
    Emp_ID     INT PRIMARY KEY,
    Emp_Name   VARCHAR(50),
    Salary     DECIMAL(10,2),
    Dept_ID    INT,
    Manager_ID INT           -- NULL = top of the hierarchy
);
INSERT INTO Department VALUES
    (1, 'Computer Science', 'Block A'),
    (2, 'Mathematics',      'Block B'),
    (3, 'Physics',          'Block C'),
    (4, 'Commerce',         'Block D');       -- has no students
INSERT INTO Course VALUES
    (1, 'Data Structures',   '6 Months', 15000, 1),
    (2, 'DBMS',              '4 Months', 12000, 1),
    (3, 'Calculus',          '5 Months',  9000, 2),
    (4, 'Quantum Mechanics', '6 Months', 14000, 3),
    (5, 'Machine Learning',  '8 Months', 25000, 1);   -- has no students
INSERT INTO Student VALUES
    (101, 'Rahul Sharma', 20, 'Meerut',   1,    1),
    (102, 'Priya Singh',  21, 'Delhi',    1,    2),
    (103, 'Amit Verma',   22, 'Roorkee',  2,    3),
    (104, 'Neha Gupta',   19, 'Meerut',   1,    1),
    (105, 'Karan Mehta',  23, 'Dehradun', 3,    4),
    (106, 'Sneha Joshi',  20, 'Haridwar', NULL, NULL),
    (107, 'Vikas Kumar',  21, 'Delhi',    2,    3),
    (108, 'Anjali Rao',   22, 'Noida',    NULL, 2);
INSERT INTO Employee VALUES
    (1, 'Arun Kapoor',  61000, 1, NULL),
    (2, 'Meena Iyer',   42000, 2, 1),
    (3, 'Suresh Patel', 48000, 1, 1),
    (4, 'Divya Menon',  55000, 1, 3),
    (5, 'Ravi Das',     30000, 3, 2),
    (6, 'Rekha Nair',   45000, 2, 2),
    (7, 'Sanjay Roy',   52000, 3, 1);
SELECT * FROM Department;
SELECT * FROM Course;
SELECT * FROM Student;
SELECT * FROM Employee;
```

Output

```
Query OK, 0 rows affected

Query OK, 0 rows affected

Query OK, 0 rows affected

Query OK, 0 rows affected

Query OK, 4 rows affected
Records: 4  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected
Records: 5  Duplicates: 0  Warnings: 0

Query OK, 8 rows affected
Records: 8  Duplicates: 0  Warnings: 0

Query OK, 7 rows affected
Records: 7  Duplicates: 0  Warnings: 0

+---------+-----------------+----------+
| Dept_ID | Dept_Name       | Location |
+---------+-----------------+----------+
|       1 | Computer Science | Block A  |
|       2 | Mathematics      | Block B  |
|       3 | Physics          | Block C  |
|       4 | Commerce         | Block D  |
+---------+-----------------+----------+
4 rows in set

+-----------+-------------------+----------+----------+---------+
| Course_ID | Course_Name       | Duration | Fees     | Dept_ID |
+-----------+-------------------+----------+----------+---------+
|         1 | Data Structures   | 6 Months | 15000.00 |       1 |
|         2 | DBMS              | 4 Months | 12000.00 |       1 |
|         3 | Calculus          | 5 Months |  9000.00 |       2 |
|         4 | Quantum Mechanics | 6 Months | 14000.00 |       3 |
|         5 | Machine Learning  | 8 Months | 25000.00 |       1 |
+-----------+-------------------+----------+----------+---------+
5 rows in set

+------------+--------------+------+----------+---------+-----------+
| Student_ID | Name         | Age  | City     | Dept_ID | Course_ID |
+------------+--------------+------+----------+---------+-----------+
|        101 | Rahul Sharma |   20 | Meerut   |       1 |         1 |
|        102 | Priya Singh  |   21 | Delhi    |       1 |         2 |
|        103 | Amit Verma   |   22 | Roorkee  |       2 |         3 |
|        104 | Neha Gupta   |   19 | Meerut   |       1 |         1 |
|        105 | Karan Mehta  |   23 | Dehradun |       3 |         4 |
|        106 | Sneha Joshi  |   20 | Haridwar |    NULL |      NULL |
|        107 | Vikas Kumar  |   21 | Delhi    |       2 |         3 |
|        108 | Anjali Rao   |   22 | Noida    |    NULL |         2 |
+------------+--------------+------+----------+---------+-----------+
8 rows in set

+--------+--------------+----------+---------+------------+
| Emp_ID | Emp_Name     | Salary   | Dept_ID | Manager_ID |
+--------+--------------+----------+---------+------------+
|      1 | Arun Kapoor  | 61000.00 |       1 |       NULL |
|      2 | Meena Iyer   | 42000.00 |       2 |          1 |
|      3 | Suresh Patel | 48000.00 |       1 |          1 |
|      4 | Divya Menon  | 55000.00 |       1 |          3 |
|      5 | Ravi Das     | 30000.00 |       3 |          2 |
|      6 | Rekha Nair   | 45000.00 |       2 |          2 |
|      7 | Sanjay Roy   | 52000.00 |       3 |          1 |
+--------+--------------+----------+---------+------------+
7 rows in set
```

A. INNER JOIN

Q1. Demonstrate an INNER JOIN between Student and Department.

SQL

```
SELECT s.Student_ID, s.Name, d.Dept_ID, d.Dept_Name
FROM Student s
INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+------------+--------------+---------+------------------+
| Student_ID | Name         | Dept_ID | Dept_Name        |
+------------+--------------+---------+------------------+
|        101 | Rahul Sharma |       1 | Computer Science |
|        102 | Priya Singh  |       1 | Computer Science |
|        103 | Amit Verma   |       2 | Mathematics      |
|        104 | Neha Gupta   |       1 | Computer Science |
|        105 | Karan Mehta  |       3 | Physics          |
|        107 | Vikas Kumar  |       2 | Mathematics      |
+------------+--------------+---------+------------------+
6 rows in set
```

Q2. Display student names along with their department names using INNER JOIN.

SQL

```
SELECT s.Name AS Student_Name, d.Dept_Name AS Department
FROM Student s
INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+--------------+------------------+
| Student_Name | Department       |
+--------------+------------------+
| Rahul Sharma | Computer Science |
| Priya Singh  | Computer Science |
| Amit Verma   | Mathematics      |
| Neha Gupta   | Computer Science |
| Karan Mehta  | Physics          |
| Vikas Kumar  | Mathematics      |
+--------------+------------------+
6 rows in set
```

Q3. Display employee names along with their department names.

SQL

```
SELECT e.Emp_Name, d.Dept_Name
FROM Employee e
INNER JOIN Department d ON e.Dept_ID = d.Dept_ID;
```

Output

```
+--------------+------------------+
| Emp_Name     | Dept_Name        |
+--------------+------------------+
| Arun Kapoor  | Computer Science |
| Meena Iyer   | Mathematics      |
| Suresh Patel | Computer Science |
| Divya Menon  | Computer Science |
| Ravi Das     | Physics          |
| Rekha Nair   | Mathematics      |
| Sanjay Roy   | Physics          |
+--------------+------------------+
7 rows in set
```

Q4. Display students along with their course details.

SQL

```
SELECT s.Name, c.Course_ID, c.Course_Name, c.Duration, c.Fees
FROM Student s
INNER JOIN Course c ON s.Course_ID = c.Course_ID;
```

Output

```
+--------------+-----------+-------------------+----------+----------+
| Name         | Course_ID | Course_Name       | Duration | Fees     |
+--------------+-----------+-------------------+----------+----------+
| Rahul Sharma |         1 | Data Structures   | 6 Months | 15000.00 |
| Priya Singh  |         2 | DBMS              | 4 Months | 12000.00 |
| Amit Verma   |         3 | Calculus          | 5 Months |  9000.00 |
| Neha Gupta   |         1 | Data Structures   | 6 Months | 15000.00 |
| Karan Mehta  |         4 | Quantum Mechanics | 6 Months | 14000.00 |
| Vikas Kumar  |         3 | Calculus          | 5 Months |  9000.00 |
| Anjali Rao   |         2 | DBMS              | 4 Months | 12000.00 |
+--------------+-----------+-------------------+----------+----------+
7 rows in set
```

Q5. Find students belonging to a particular department using INNER JOIN.

SQL

```
SELECT s.Name, s.City, d.Dept_Name
FROM Student s
INNER JOIN Department d ON s.Dept_ID = d.Dept_ID
WHERE d.Dept_Name = 'Computer Science';
```

Output

```
+--------------+--------+------------------+
| Name         | City   | Dept_Name        |
+--------------+--------+------------------+
| Rahul Sharma | Meerut | Computer Science |
| Priya Singh  | Delhi  | Computer Science |
| Neha Gupta   | Meerut | Computer Science |
+--------------+--------+------------------+
3 rows in set
```

B. OUTER JOINS

Q6. Demonstrate a LEFT OUTER JOIN between Student and Department.

SQL

```
SELECT s.Name, d.Dept_Name
FROM Student s
LEFT OUTER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+--------------+------------------+
| Name         | Dept_Name        |
+--------------+------------------+
| Rahul Sharma | Computer Science |
| Priya Singh  | Computer Science |
| Amit Verma   | Mathematics      |
| Neha Gupta   | Computer Science |
| Karan Mehta  | Physics          |
| Sneha Joshi  | NULL             |
| Vikas Kumar  | Mathematics      |
| Anjali Rao   | NULL             |
+--------------+------------------+
8 rows in set
```

Q7. Display all students, including those who do not have a matching department.

SQL

```
-- unmatched students show NULL in Dept_Name
SELECT s.Student_ID, s.Name, d.Dept_Name
FROM Student s
LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
ORDER BY s.Student_ID;
```

Output

```
+------------+--------------+------------------+
| Student_ID | Name         | Dept_Name        |
+------------+--------------+------------------+
|        101 | Rahul Sharma | Computer Science |
|        102 | Priya Singh  | Computer Science |
|        103 | Amit Verma   | Mathematics      |
|        104 | Neha Gupta   | Computer Science |
|        105 | Karan Mehta  | Physics          |
|        106 | Sneha Joshi  | NULL             |
|        107 | Vikas Kumar  | Mathematics      |
|        108 | Anjali Rao   | NULL             |
+------------+--------------+------------------+
8 rows in set
```

Q8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

SQL

```
SELECT s.Name, d.Dept_Name
FROM Student s
RIGHT OUTER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+--------------+------------------+
| Name         | Dept_Name        |
+--------------+------------------+
| Rahul Sharma | Computer Science |
| Priya Singh  | Computer Science |
| Amit Verma   | Mathematics      |
| Neha Gupta   | Computer Science |
| Karan Mehta  | Physics          |
| Vikas Kumar  | Mathematics      |
| NULL         | Commerce         |
+--------------+------------------+
7 rows in set
```

Q9. Display all departments, including departments having no students.

SQL

```
SELECT d.Dept_ID, d.Dept_Name, s.Name AS Student
FROM Department d
LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
ORDER BY d.Dept_ID;
```

Output

```
+---------+------------------+--------------+
| Dept_ID | Dept_Name        | Student      |
+---------+------------------+--------------+
|       1 | Computer Science | Rahul Sharma |
|       1 | Computer Science | Priya Singh  |
|       1 | Computer Science | Neha Gupta   |
|       2 | Mathematics      | Vikas Kumar  |
|       2 | Mathematics      | Amit Verma   |
|       3 | Physics          | Karan Mehta  |
|       4 | Commerce         | NULL         |
+---------+------------------+--------------+
7 rows in set
```

Q10. Find students who are not assigned to any department using LEFT JOIN.

SQL

```
SELECT s.Student_ID, s.Name
FROM Student s
LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
WHERE d.Dept_ID IS NULL;
```

Output

```
+------------+-------------+
| Student_ID | Name        |
+------------+-------------+
|        106 | Sneha Joshi |
|        108 | Anjali Rao  |
+------------+-------------+
2 rows in set
```

Q11. Find departments having no students using RIGHT/LEFT JOIN.

SQL

```
-- using RIGHT JOIN
SELECT d.Dept_ID, d.Dept_Name
FROM Student s
RIGHT JOIN Department d ON s.Dept_ID = d.Dept_ID
WHERE s.Student_ID IS NULL;

-- same result using LEFT JOIN
SELECT d.Dept_ID, d.Dept_Name
FROM Department d
LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
WHERE s.Student_ID IS NULL;
```

Output

```
+---------+-----------+
| Dept_ID | Dept_Name |
+---------+-----------+
|       4 | Commerce  |
+---------+-----------+
1 row in set

+---------+-----------+
| Dept_ID | Dept_Name |
+---------+-----------+
|       4 | Commerce  |
+---------+-----------+
1 row in set
```

Q12. Demonstrate a FULL OUTER JOIN using a workaround (MySQL has no FULL JOIN keyword).

SQL

```
-- LEFT JOIN  UNION  RIGHT JOIN = FULL OUTER JOIN
SELECT s.Name, d.Dept_Name
FROM Student s LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
UNION
SELECT s.Name, d.Dept_Name
FROM Student s RIGHT JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+--------------+------------------+
| Name         | Dept_Name        |
+--------------+------------------+
| Rahul Sharma | Computer Science |
| Priya Singh  | Computer Science |
| Amit Verma   | Mathematics      |
| Neha Gupta   | Computer Science |
| Karan Mehta  | Physics          |
| Sneha Joshi  | NULL             |
| Vikas Kumar  | Mathematics      |
| Anjali Rao   | NULL             |
| NULL         | Commerce         |
+--------------+------------------+
9 rows in set
```

C. CROSS and NATURAL JOIN

Q13. Demonstrate a CROSS JOIN between Student and Course.

SQL

```
SELECT s.Name, c.Course_Name
FROM Student s
CROSS JOIN Course c
ORDER BY s.Student_ID, c.Course_ID;
```

Output

```
+--------------+------------------+
| Name         | Course_Name      |
+--------------+------------------+
| Rahul Sharma | Data Structures  |
| Rahul Sharma | DBMS             |
| Rahul Sharma | Calculus         |
| Rahul Sharma | Quantum Mechanics |
| Rahul Sharma | Machine Learning |
| Priya Singh  | Data Structures  |
| Priya Singh  | DBMS             |
| Priya Singh  | Calculus         |
| Priya Singh  | Quantum Mechanics |
| Priya Singh  | Machine Learning |
| Amit Verma   | Data Structures  |
| Amit Verma   | DBMS             |
| Amit Verma   | Calculus         |
| Amit Verma   | Quantum Mechanics |
| Amit Verma   | Machine Learning |
| Neha Gupta   | Data Structures  |
| Neha Gupta   | DBMS             |
| Neha Gupta   | Calculus         |
| Neha Gupta   | Quantum Mechanics |
| Neha Gupta   | Machine Learning |
| Karan Mehta  | Data Structures  |
| Karan Mehta  | DBMS             |
| Karan Mehta  | Calculus         |
| Karan Mehta  | Quantum Mechanics |
| Karan Mehta  | Machine Learning |
| Sneha Joshi  | Data Structures  |
| Sneha Joshi  | DBMS             |
| Sneha Joshi  | Calculus         |
| Sneha Joshi  | Quantum Mechanics |
| Sneha Joshi  | Machine Learning |
| Vikas Kumar  | Data Structures  |
| Vikas Kumar  | DBMS             |
| Vikas Kumar  | Calculus         |
| Vikas Kumar  | Quantum Mechanics |
| Vikas Kumar  | Machine Learning |
| Anjali Rao   | Data Structures  |
| Anjali Rao   | DBMS             |
| Anjali Rao   | Calculus         |
| Anjali Rao   | Quantum Mechanics |
| Anjali Rao   | Machine Learning |
+--------------+------------------+
40 rows in set
```

Q14. Calculate the number of possible Student-Course combinations using CROSS JOIN.

SQL

```
-- rows in a cross join = students x courses
SELECT COUNT(*) AS Total_Combinations,
       (SELECT COUNT(*) FROM Student) AS Students,
       (SELECT COUNT(*) FROM Course)  AS Courses
FROM Student CROSS JOIN Course;
```

Output

```
+--------------------+----------+---------+
| Total_Combinations | Students | Courses |
+--------------------+----------+---------+
|                 40 |        8 |       5 |
+--------------------+----------+---------+
1 row in set
```

Q15. Demonstrate a NATURAL JOIN between two suitable tables.

SQL

```
-- Student and Department share only the column Dept_ID
SELECT * FROM Student NATURAL JOIN Department;
```

Output

```
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
| Dept_ID | Student_ID | Name         | Age  | City     | Course_ID | Dept_Name       | Location |
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
|       1 |        101 | Rahul Sharma |   20 | Meerut   |         1 | Computer Science | Block A  |
|       1 |        102 | Priya Singh  |   21 | Delhi    |         2 | Computer Science | Block A  |
|       2 |        103 | Amit Verma   |   22 | Roorkee  |         3 | Mathematics      | Block B  |
|       1 |        104 | Neha Gupta   |   19 | Meerut   |         1 | Computer Science | Block A  |
|       3 |        105 | Karan Mehta  |   23 | Dehradun |         4 | Physics          | Block C  |
|       2 |        107 | Vikas Kumar  |   21 | Delhi    |         3 | Mathematics      | Block B  |
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
6 rows in set
```

Q16. Compare the result of NATURAL JOIN and INNER JOIN.

SQL

```
-- NATURAL JOIN: Dept_ID appears once (merged column)
SELECT * FROM Student NATURAL JOIN Department;

-- INNER JOIN ... ON: Dept_ID appears twice (once from each table)
SELECT * FROM Student s INNER JOIN Department d ON s.Dept_ID = d.Dept_ID;
```

Output

```
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
| Dept_ID | Student_ID | Name         | Age  | City     | Course_ID | Dept_Name       | Location |
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
|       1 |        101 | Rahul Sharma |   20 | Meerut   |         1 | Computer Science | Block A  |
|       1 |        102 | Priya Singh  |   21 | Delhi    |         2 | Computer Science | Block A  |
|       2 |        103 | Amit Verma   |   22 | Roorkee  |         3 | Mathematics      | Block B  |
|       1 |        104 | Neha Gupta   |   19 | Meerut   |         1 | Computer Science | Block A  |
|       3 |        105 | Karan Mehta  |   23 | Dehradun |         4 | Physics          | Block C  |
|       2 |        107 | Vikas Kumar  |   21 | Delhi    |         3 | Mathematics      | Block B  |
+---------+------------+--------------+------+----------+-----------+-----------------+----------+
6 rows in set

+------------+--------------+------+----------+---------+-----------+---------+-----------------+----------+
| Student_ID | Name         | Age  | City     | Dept_ID | Course_ID | Dept_ID | Dept_Name       | Location |
+------------+--------------+------+----------+---------+-----------+---------+-----------------+----------+
|        101 | Rahul Sharma |   20 | Meerut   |       1 |         1 |       1 | Computer Science | Block A  |
|        102 | Priya Singh  |   21 | Delhi    |       1 |         2 |       1 | Computer Science | Block A  |
|        103 | Amit Verma   |   22 | Roorkee  |       2 |         3 |       2 | Mathematics      | Block B  |
|        104 | Neha Gupta   |   19 | Meerut   |       1 |         1 |       1 | Computer Science | Block A  |
|        105 | Karan Mehta  |   23 | Dehradun |       3 |         4 |       3 | Physics          | Block C  |
|        107 | Vikas Kumar  |   21 | Delhi    |       2 |         3 |       2 | Mathematics      | Block B  |
+------------+--------------+------+----------+---------+-----------+---------+-----------------+----------+
6 rows in set
```

D. SELF JOIN and multi-table joins

Q17. Perform a SELF JOIN on the Employee table to show employee-manager relationships.

SQL

```
SELECT e.Emp_ID, e.Emp_Name AS Employee, m.Emp_ID AS Manager_ID, m.Emp_Name AS Manager
FROM Employee e
INNER JOIN Employee m ON e.Manager_ID = m.Emp_ID;
```

Output

```
+--------+--------------+------------+--------------+
| Emp_ID | Employee     | Manager_ID | Manager      |
+--------+--------------+------------+--------------+
|      2 | Meena Iyer   |          1 | Arun Kapoor  |
|      3 | Suresh Patel |          1 | Arun Kapoor  |
|      4 | Divya Menon  |          3 | Suresh Patel |
|      5 | Ravi Das     |          2 | Meena Iyer   |
|      6 | Rekha Nair   |          2 | Meena Iyer   |
|      7 | Sanjay Roy   |          1 | Arun Kapoor  |
+--------+--------------+------------+--------------+
6 rows in set
```

Q18. Display employees along with their managers using SELF JOIN.

SQL

```
-- LEFT JOIN so the top employee (no manager) is also listed
SELECT e.Emp_Name AS Employee, m.Emp_Name AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.Manager_ID = m.Emp_ID
ORDER BY e.Emp_ID;
```

Output

```
+--------------+--------------+
| Employee     | Manager      |
+--------------+--------------+
| Arun Kapoor  | NULL         |
| Meena Iyer   | Arun Kapoor  |
| Suresh Patel | Arun Kapoor  |
| Divya Menon  | Suresh Patel |
| Ravi Das     | Meena Iyer   |
| Rekha Nair   | Meena Iyer   |
| Sanjay Roy   | Arun Kapoor  |
+--------------+--------------+
7 rows in set
```

Q19. Join three tables - Student, Department and Course.

SQL

```
SELECT s.Name, d.Dept_Name, c.Course_Name
FROM Student s
INNER JOIN Department d ON s.Dept_ID = d.Dept_ID
INNER JOIN Course c     ON s.Course_ID = c.Course_ID;
```

Output

```
+--------------+------------------+--------------------+
| Name         | Dept_Name        | Course_Name        |
+--------------+------------------+--------------------+
| Rahul Sharma | Computer Science | Data Structures    |
| Priya Singh  | Computer Science | DBMS               |
| Amit Verma   | Mathematics      | Calculus           |
| Neha Gupta   | Computer Science | Data Structures    |
| Karan Mehta  | Physics          | Quantum Mechanics  |
| Vikas Kumar  | Mathematics      | Calculus           |
+--------------+------------------+--------------------+
6 rows in set
```

Q20. Display student name, course name and department name using multiple joins.

SQL

```
SELECT s.Name AS Student, c.Course_Name AS Course, d.Dept_Name AS Department
FROM Student s
LEFT JOIN Course c     ON s.Course_ID = c.Course_ID
LEFT JOIN Department d ON s.Dept_ID = d.Dept_ID
ORDER BY s.Student_ID;
```

Output

```
+--------------+--------------------+------------------+
| Student      | Course             | Department       |
+--------------+--------------------+------------------+
| Rahul Sharma | Data Structures    | Computer Science |
| Priya Singh  | DBMS               | Computer Science |
| Amit Verma   | Calculus           | Mathematics      |
| Neha Gupta   | Data Structures    | Computer Science |
| Karan Mehta  | Quantum Mechanics  | Physics          |
| Sneha Joshi  | NULL               | NULL             |
| Vikas Kumar  | Calculus           | Mathematics      |
| Anjali Rao   | DBMS               | NULL             |
+--------------+--------------------+------------------+
8 rows in set
```

E. Joins with aggregation and subqueries

Q21. Find employees whose salary is greater than the average salary of their department.

SQL

```
SELECT e.Emp_Name, e.Dept_ID, e.Salary, ROUND(a.Avg_Salary, 2) AS Dept_Avg_Salary
FROM Employee e
INNER JOIN (SELECT Dept_ID, AVG(Salary) AS Avg_Salary
            FROM Employee
            GROUP BY Dept_ID) a ON e.Dept_ID = a.Dept_ID
WHERE e.Salary > a.Avg_Salary;
```

Output

```
+-------------+---------+----------+-----------------+
| Emp_Name    | Dept_ID | Salary   | Dept_Avg_Salary |
+-------------+---------+----------+-----------------+
| Arun Kapoor |       1 | 61000.00 |        54666.67 |
| Divya Menon |       1 | 55000.00 |        54666.67 |
| Rekha Nair  |       2 | 45000.00 |        43500.00 |
| Sanjay Roy  |       3 | 52000.00 |        41000.00 |
+-------------+---------+----------+-----------------+
4 rows in set
```

Q22. Find the department having the maximum number of students.

SQL

```
SELECT d.Dept_Name, COUNT(s.Student_ID) AS Total_Students
FROM Department d
INNER JOIN Student s ON s.Dept_ID = d.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name
ORDER BY Total_Students DESC
LIMIT 1;
```

Output

```
+------------------+----------------+
| Dept_Name        | Total_Students |
+------------------+----------------+
| Computer Science |              3 |
+------------------+----------------+
1 row in set
```

Q23. Display departments and the total number of students in each department.

SQL

```
SELECT d.Dept_Name, COUNT(s.Student_ID) AS Total_Students
FROM Department d
LEFT JOIN Student s ON s.Dept_ID = d.Dept_ID
GROUP BY d.Dept_ID, d.Dept_Name
ORDER BY d.Dept_ID;
```

Output

```
+------------------+----------------+
| Dept_Name        | Total_Students |
+------------------+----------------+
| Computer Science |              3 |
| Mathematics      |              2 |
| Physics          |              1 |
| Commerce         |              0 |
+------------------+----------------+
4 rows in set
```

Q24. Display courses along with the number of students enrolled in each course.

SQL

```
SELECT c.Course_Name, COUNT(s.Student_ID) AS Students_Enrolled
FROM Course c
LEFT JOIN Student s ON s.Course_ID = c.Course_ID
GROUP BY c.Course_ID, c.Course_Name
ORDER BY c.Course_ID;
```

Output

```
+------------------+------------------+
| Course_Name      | Students_Enrolled |
+------------------+------------------+
| Data Structures  |                2 |
| DBMS             |                2 |
| Calculus         |                2 |
| Quantum Mechanics |               1 |
| Machine Learning |                0 |
+------------------+------------------+
5 rows in set
```

Q25. Complete student-department-course report using INNER JOIN, LEFT JOIN, GROUP BY and aggregates.

SQL

```
SELECT d.Dept_Name,
       c.Course_Name,
       COUNT(s.Student_ID)             AS Students_Enrolled,
       ROUND(AVG(s.Age), 1)            AS Avg_Age,
       c.Fees * COUNT(s.Student_ID)    AS Fee_Collected
FROM Course c
INNER JOIN Department d ON c.Dept_ID = d.Dept_ID
LEFT JOIN Student s     ON s.Course_ID = c.Course_ID
GROUP BY d.Dept_Name, c.Course_ID, c.Course_Name, c.Fees
ORDER BY d.Dept_Name, c.Course_Name;
```

Output

```
+-----------------+------------------+------------------+---------+--------------+
| Dept_Name       | Course_Name      | Students_Enrolled | Avg_Age | Fee_Collected |
+-----------------+------------------+------------------+---------+--------------+
| Computer Science | Data Structures |                2 |    19.5 |      30000.00 |
| Computer Science | DBMS            |                2 |    21.5 |      24000.00 |
| Computer Science | Machine Learning |               0 |    NULL |          0.00 |
| Mathematics     | Calculus         |                2 |    21.5 |      18000.00 |
| Physics         | Quantum Mechanics |               1 |    23.0 |      14000.00 |
+-----------------+------------------+------------------+---------+--------------+
5 rows in set
```